

Capitolo 49 — PROT-020 — WCM

Technical Issue Tracking V1

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-49
Data master	2026-09-02
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/49_prot_020_wcm_technical_issue_tracking_v1.md
Source SHA	76696d7
Release	2026-09-04T09:44:14.164Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE VII — Il Libro dei Protocolli WCM Stato: FROZEN **Data:** 2026-09-02 **Scope:** WCM generale, domain-agnostic

49.0 Fermarsi in sicurezza non basta: bisogna lasciare una traccia

Un sistema affidabile deve saper dire «non posso continuare» quando una condizione tecnica obbligatoria non è soddisfatta.

Nel WCM questo principio compare spesso con l'espressione **fail closed**: se una verifica indispensabile fallisce, il sistema non deve inventare un risultato positivo, non deve ignorare l'errore e non deve proseguire come se nulla fosse.

Ma c'è un secondo problema.

Se il sistema si ferma correttamente e poi non lascia una traccia durevole del motivo, la sicurezza è stata preservata solo a metà. Una nuova sessione, un altro operatore o un altro componente potrebbe non sapere più:

- che cosa è fallito;
- dove è fallito;
- quale evidenza ha prodotto il blocco;
- se il problema è ancora aperto;
- che cosa è stato fatto per risolverlo.

PROT-020 nasce per chiudere questo vuoto.

Il suo principio fondamentale è:

FAIL CLOSED ≠ FAIL SILENT

In altre parole: il WCM può e deve fermarsi quando una verifica tecnica deterministica fallisce, ma deve anche rendere quel fallimento osservabile, persistente e recuperabile.

PROT-020 non trasforma il WCM in un software di ticketing generale. Introduce invece una capacità più piccola e precisa: registrare una **technical issue** quando una failure meccanica, strutturata e riproducibile impedisce una transizione o una proiezione prevista.

49.1 Che cosa significa “technical issue” nel WCM

Nel linguaggio comune, la parola “problema” può indicare quasi tutto: una decisione difficile, una regola poco chiara, una contraddizione, un errore umano, un dubbio strategico o un malfunzionamento tecnico.

PROT-020 restringe volutamente il campo.

Una technical issue V1 riguarda una **failure tecnica deterministica bloccante**.

Le tre parole sono importanti.

Tecnica significa che il problema appartiene al funzionamento del sistema, non al significato di una decisione.

Deterministica significa che il fallimento può essere riconosciuto attraverso una regola o un controllo preciso, senza dover interpretare semanticamente la situazione.

Bloccante significa che quella failure impedisce una transizione o una proiezione che il sistema avrebbe dovuto completare.

Un esempio astratto può chiarire la differenza.

Immaginiamo che una procedura debba leggere un oggetto strutturato che contiene cinque campi obbligatori. Uno di quei campi manca. Il validatore lo rileva e rifiuta l’oggetto.

Questo è un problema tecnico deterministico: la regola è esplicita, il controllo è riproducibile e il sistema sa esattamente perché non può procedere.

Diverso sarebbe il caso in cui il documento esista ma il suo significato sia ambiguo. In quel caso non basta una technical issue per decidere cosa sia corretto: serve una valutazione cognitiva o un’autorità appropriata.

PROT-020 non confonde questi due piani.

49.2 Perché il semplice log non è sufficiente

Molti sistemi producono log.

Un log può essere molto utile: registra messaggi, errori, timestamp, eventi e dettagli tecnici. Ma un log non è automaticamente una memoria operativa strutturata.

Il problema è che un errore perso in centinaia o migliaia di righe può essere difficile da recuperare e ancora più difficile da trattare come oggetto con un proprio ciclo di vita.

PROT-020 introduce quindi una distinzione:

LOG

→ descrive eventi tecnici

TECHNICAL ISSUE

→ rappresenta un problema tecnico persistente con identità e stato

La technical issue non sostituisce i log. Li completa quando il fallimento è abbastanza importante da impedire il normale avanzamento del sistema.

Il vantaggio è che il problema smette di essere soltanto “qualcosa che è successo” e diventa “qualcosa che è ancora aperto oppure è stato chiuso con una precisa evidenza di risoluzione”.

49.3 Una issue è un oggetto persistente

La baseline corrente persiste ogni technical issue in un oggetto dedicato del runtime del progetto.

Il percorso tecnico previsto è:

```
projects/<project-id>/runtime/technical-issues/WCM-ISSUE-*.json
```

Per un lettore non tecnico, il concetto importante non è il formato del file.

Conta il fatto che la issue:

- sopravvive alla sessione corrente;
- possiede un'identità stabile;
- appartiene a un progetto preciso;
- conserva la fonte del problema;
- espone uno stato;
- può essere riletta da altri componenti;
- resta nello storico anche dopo la chiusura.

Questo la rende parte della Persistent Organizational Memory relativa al funzionamento operativo del sistema.

La conversazione può descrivere il problema. La technical issue lo rende durevole.

49.4 Identità stabile: lo stesso problema non deve diventare dieci problemi

PROT-020 assegna a ogni issue un identificatore stabile nel formato:

```
WCM-ISSUE-YYYYMMDD-XXXXXXXXXX
```

La forma esatta dell'identificatore è meno importante del principio.

L'identità deve dipendere dall'evidenza tecnica, non dal modo in cui una persona o un'interfaccia decide di descriverla.

Questo evita una situazione comune nei sistemi poco strutturati: lo stesso problema viene rilevato più volte, con parole leggermente diverse, e finisce per sembrare una collezione di problemi differenti.

Un'identità stabile rende possibili:

- deduplicazione;
- correlazione tra rilevazioni successive;
- chiusura dell'oggetto corretto;

- audit dello storico;
- proiezioni coerenti verso interfacce diverse.

La UI può cambiare titolo o presentazione. L'identità operativa non dovrebbe cambiare con essa.

49.5 Che cosa deve sapere il sistema di una issue

La baseline V1 definisce un insieme minimo di informazioni.

In forma semplificata, una technical issue deve poter rispondere a queste domande:

1. **qual è la sua identità?**
2. **a quale progetto appartiene?**
3. **che tipo di problema è?**
4. **è aperta o chiusa?**
5. **blocca realmente qualcosa?**
6. **chi o quale componente l'ha rilevata?**
7. **quando è stata rilevata?**
8. **qual è il codice di errore?**
9. **qual è il dettaglio tecnico?**
10. **qual è la fonte esatta da cui deriva?**
11. **qual era la versione della fonte al momento della rilevazione?**
12. **se è chiusa, quando, da chi e con quale nota di risoluzione?**

La versione tecnica di questi dati usa campi strutturati come `issue_id`, `project_id`, `status`, `error_code`, `source_path`, `source_sha`, `closed_at` e `resolution_note`.

Il significato organizzativo è più importante del formato: una issue deve essere abbastanza precisa da poter essere ricostruita senza affidarsi alla memoria di chi l'ha vista per primo.

49.6 Quando una technical issue può essere aperta

PROT-020 non autorizza ad aprire una issue per qualsiasi anomalia.

La baseline richiede quattro condizioni.

1. Esiste un errore strutturato e riproducibile

Il problema deve poter essere osservato nuovamente applicando lo stesso controllo alle stesse condizioni rilevanti.

Non basta una sensazione del tipo “qualcosa sembra strano”.

2. La failure blocca una transizione o una proiezione prevista

L'anomalia deve avere un effetto operativo reale.

Un dettaglio cosmetico o un warning innocuo non diventa automaticamente technical issue bloccante.

3. Fonte e versione sono identificabili

Il sistema deve sapere dove nasce il problema e quale versione della fonte lo ha prodotto. Questo protegge la lineage dell'evidenza.

4. Non serve interpretazione semantica per sapere che il controllo è fallito

Il sistema può aprire la issue quando la failure è meccanicamente determinabile.

Se invece bisogna decidere il significato di una regola, scegliere tra alternative o interpretare una contraddizione, PROT-020 non crea authority dove non esiste.

49.7 Aprire una issue non significa modificare il workflow

Questa distinzione è essenziale.

La technical issue osserva e registra un problema. Non sostituisce il workflow che ha incontrato quel problema.

L'apertura della issue:

- **non modifica automaticamente il workflow;**
- **non crea authority;**
- **non crea automaticamente una richiesta di intervento umano;**
- **non rende valida la transizione che era fallita.**

Possiamo rappresentarlo così:

```
CONTROLLO FALLISCE
  ↓
FAIL CLOSED
  ↓
TECHNICAL ISSUE OPEN
  ↓
TRACCIA DUREVOLE DEL PROBLEMA

ma

ISSUE OPEN
≠ WORKFLOW CORRETTO
≠ TRANSIZIONE COMPLETATA
≠ AUTHORITY CONCESSA
```

La issue è una memoria del problema, non un permesso per aggirarlo.

49.8 Technical issue non significa automaticamente “serve una persona”

Quando un sistema incontra un problema, la tentazione più semplice è notificare un essere umano.

Ma un’organizzazione intelligente dovrebbe prima distinguere ciò che richiede davvero una decisione da ciò che richiede soltanto una riparazione tecnica.

PROT-020 stabilisce quindi che l’apertura di una issue non crea automaticamente un **Need** umano.

Questo protegge l’attenzione delle persone.

Un errore tecnico può essere serio senza richiedere una decisione strategica. Può essere necessario correggere un file malformato, riallineare una proiezione, ripristinare una pipeline o rieseguire un controllo.

Solo se la risoluzione supera il perimetro tecnico e richiede authority o una decisione, allora entreranno in gioco i gate appropriati.

La technical issue, da sola, non inventa quel passaggio.

49.9 La projection: rendere visibile la issue senza spostare la verità

Una issue persistita nel runtime deve poter essere mostrata a chi governa il sistema.

La baseline corrente prevede una pipeline di proiezione indipendente:

```
issue JSON
→ validazione deterministica
→ Technical Issue Projector
→ read-model
→ Mission Control / area issue
```

Il principio è lo stesso già incontrato parlando di altre proiezioni WCM: l’interfaccia è una vista della realtà persistita, non la sua fonte originaria.

La source of truth rimane l’oggetto persistente.

Questo significa che se l’interfaccia non mostra temporaneamente la issue, la issue non cessa di esistere. E se una UI presenta un dato diverso dalla fonte, non è la UI a riscrivere automaticamente la realtà.

La projection deve inoltre essere ledger/upsert-only: una issue che non compare in un aggiornamento non viene cancellata semplicemente per omissione.

È un’altra forma di protezione contro la perdita silenziosa di memoria operativa.

49.10 OPEN e CLOSED: un ciclo volutamente semplice

La V1 usa soltanto due stati:

```
OPEN  
CLOSED
```

Questa semplicità è intenzionale.

OPEN significa che la failure tecnica è ancora considerata irrisolta secondo i criteri del protocollo.

CLOSED significa che esistono le condizioni richieste per dichiarare tecnicamente risolto quel problema.

Non ci sono, in V1, stati come “assegnato”, “in analisi”, “in lavorazione”, “in attesa”, “risolto parzialmente” o “rifiutato”.

PROT-020 non vuole costruire un workflow di ticketing completo dentro il WCM. Vuole soltanto assicurare che un problema tecnico bloccante non scompaia e che la sua chiusura sia verificabile.

49.11 La chiusura è manuale tecnica

Una issue non si chiude automaticamente solo perché il controllo successivo è tornato verde.

La baseline V1 richiede una **chiusura manuale tecnica**.

Prima della transizione **OPEN** → **CLOSED** devono esistere almeno:

- il repair applicato;
- il controllo deterministico originariamente fallito ora in PASS;
- la riconciliazione o proiezione downstream verificata, quando pertinente;
- i dati di chiusura (**closed_at**, **closed_by**, **resolution_note**).

Questa scelta impedisce un auto-close troppo aggressivo.

Un controllo potrebbe tornare verde per ragioni temporanee o perché è cambiata una condizione esterna. La chiusura richiede invece che il tecnico possa dichiarare quale riparazione è stata applicata e quale evidenza dimostra il recupero.

Il principio è:

```
NON FALLISCE PIÙ  
≠ automaticamente  
ISSUE CHIUSA
```

Serve una closure esplicita e ricostruibile.

49.12 Perché la issue chiusa resta nello storico

Una technical issue **CLOSED** non viene cancellata.

Questa è una scelta importante per almeno tre ragioni.

La prima è l'auditabilità: deve essere possibile ricostruire che il problema è esistito, come è stato risolto e quando.

La seconda è l'apprendimento: una failure passata può diventare evidence per riconoscere pattern ricorrenti o migliorare il metodo.

La terza è la prevenzione della riscoperta ciclica: se un problema simile torna, il sistema può confrontarlo con casi precedenti invece di trattarlo come completamente nuovo.

La memoria del fallimento, quindi, non è un difetto da nascondere.

È una parte della memoria organizzativa.

49.13 Cosa PROT-020 non introduce

I confini della V1 sono volutamente stretti.

PROT-020 non introduce:

- assegnatari;
- priorità;
- SLA;
- thread di commenti;
- workflow di ticket;
- auto-close;
- escalation automatica verso una persona.

Queste funzioni potrebbero essere utili in futuro, ma cambierebbero il significato della capability.

Un sistema di issue tracking completo richiederebbe nuove regole: ownership, responsabilità, tempi, escalation, notifiche, magari severity e policy differenti.

La baseline corrente non presume che tutto ciò esista.

Questo è un esempio di maturità controllata: implementare ciò che serve per risolvere un problema preciso senza trasformare una capacità locale in una piattaforma molto più ampia senza una decisione esplicita.

49.14 Failure della issue stessa

Anche il sistema che registra problemi può avere problemi.

PROT-020 applica quindi il fail closed anche alla propria projection.

Una issue malformata, un identificatore non valido, una source fuori dallo scope previsto o uno stato incoerente devono impedire una proiezione considerata valida.

Il sistema non deve “aggiustare a intuito” una issue solo per riuscire a mostrarla.

Questo evita una contraddizione: usare un sistema nato per rendere osservabili failure deterministiche e poi interpretare in modo probabilistico gli stessi dati tecnici che dovrebbero essere precisi.

49.15 Se fallisce il Technical Issue Projector

La projection è un servizio downstream.

Se il projector che porta le issue verso il read-model fallisce, questo non rende valido il workflow principale e non cancella la source persistente.

La gerarchia concettuale resta:

```
SOURCE PERSISTENTE DELLA ISSUE
↓
VALIDAZIONE
↓
PROJECTION
↓
INTERFACCIA
```

Se la projection fallisce, il problema è tecnico e deve essere recuperato tecnicamente.

Ma la source rimane la base da cui ricostruire lo stato.

Questa separazione protegge il WCM dal confondere “non lo vedo nella UI” con “non esiste”.

49.16 Un esempio astratto completo

Immaginiamo un workflow che debba produrre una vista strutturata dello stato di un processo.

Prima di pubblicarla, un validatore controlla che il riferimento al documento target sia presente e appartenga allo scope corretto.

Il controllo trova invece un percorso fuori perimetro.

Il flusso corretto è:

```
VALIDAZIONE TARGET
↓
FAIL
↓
TRANSIZIONE NON COMPLETATA
↓
TECHNICAL ISSUE OPEN
↓
source_path + source_sha + error_code persistiti
↓
repair tecnico
↓
validazione ripetuta
↓
PASS
↓
reconciliation/projection verificata
↓
chiusura tecnica esplicita
↓
```

Ciò che non deve accadere è altrettanto importante:

- il validatore non deve ignorare il percorso errato;
- la issue non deve concedere authority;
- la UI non deve diventare source of truth;
- la issue non deve scomparire quando è chiusa;
- una nuova sessione non deve dipendere dalla memoria di chi ha osservato il primo errore.

49.17 Il rapporto con fail closed

PROT-020 non sostituisce il fail closed. Lo completa.

Fail closed risponde alla domanda:

| Posso continuare in sicurezza?

Technical Issue Tracking risponde invece a:

| Se non posso continuare, come faccio a non perdere il motivo del blocco?

Queste due capacità si rafforzano a vicenda.

Un fail closed senza memoria produce sicurezza momentanea ma scarsa continuità.

Una memoria delle issue senza fail closed produce osservabilità, ma potrebbe lasciare che il sistema continui nonostante il problema.

Il comportamento desiderato è:

RILEVA
→ BLOCCA QUANDO NECESSARIO
→ PERSISTI IL MOTIVO
→ RIPARA
→ VERIFICA
→ CHIUDI CON EVIDENZA
→ CONSERVA LO STORICO

49.18 Il rapporto con la Persistent Organizational Memory

La technical issue è un esempio concreto di una regola più generale del WCM: ciò che deve influenzare il comportamento futuro non può esistere soltanto nella Working Memory.

Una failure bloccante può accadere in una sessione e venire risolta in un'altra.

Per rendere possibile questa continuità servono:

- identità stabile;
- stato persistente;
- provenance;

- evidenza della fonte;
- storia della risoluzione.

PROT-020 applica quindi la Dual Memory a un dominio molto specifico: la memoria dei problemi tecnici operativi.

La Working Memory permette di ragionare sul problema.

La Persistent Organizational Memory permette al problema di sopravvivere al ragionamento corrente.

49.19 Il rapporto con l'apprendimento

Una technical issue non è automaticamente un learning.

Questo confine è importante.

Una singola failure può essere locale, accidentale o priva di valore metodologico generale.

Ma la persistenza delle issue crea evidence che il Learning Loop può utilizzare.

Per esempio, più issue simili potrebbero suggerire che:

- un determinato controllo manca troppo spesso;
- una stessa classe di errore ricorre;
- un repair dovrebbe diventare deterministico;
- un protocollo dovrebbe essere rafforzato;
- una capability presenta una fragilità sistematica.

La sequenza concettuale è:

```
TECHNICAL ISSUE  
→ EVIDENCE  
→ REVIEW COGNITIVA  
→ eventuale CANDIDATE LEARNING  
→ eventuale modifica del metodo tramite authority appropriata
```

La issue conserva il fatto.

Il Learning System decide se quel fatto contiene una lezione utile.

49.20 Perché questa distinzione rende il sistema più intelligente

Un'organizzazione che dimentica i propri errori è condannata a riscoprirli.

Un'organizzazione che registra ogni errore ma non sa distinguerlo dal rumore crea invece un archivio ingestibile.

PROT-020 occupa una posizione intermedia: registra soltanto una classe precisa di failure tecniche bloccanti, con criteri strutturati e un ciclo minimo di apertura e chiusura.

Questo rende possibile costruire nel tempo una memoria dei fallimenti tecnici senza trasformare ogni warning in un evento organizzativo importante.

La capacità è ancora V1 e in field validation. Non dimostra che ogni failure del WCM sia già classificata o gestita automaticamente.

Dimostra invece una direzione architetturale precisa: **un errore che blocca il sistema non deve scomparire con la sessione che lo ha osservato.**

49.21 Cosa protegge PROT-020

Il protocollo protegge soprattutto cinque proprietà.

1. Osservabilità

Il sistema non si limita a fermarsi: rende visibile il motivo.

2. Continuità

Il problema sopravvive alle sessioni e può essere ripreso.

3. Provenance

La issue conserva fonte e versione dell'evidenza tecnica.

4. Authority boundary

La registrazione di un problema non crea permessi o decisioni che non esistono.

5. Storia

La chiusura non cancella il fallimento: conserva l'esperienza.

49.22 Errori che il protocollo vuole evitare

PROT-020 nasce anche per impedire una serie di anti-pattern:

```
FAILURE TECNICA
→ messaggio in chat
→ sessione finisce
→ problema dimenticato
```

oppure:

```
FAILURE TECNICA
→ issue aperta
→ issue trattata come authority
```

oppure:

```
CONTROLLO TORNA PASS
→ auto-close senza repair verificato
```

oppure:

ISSUE NON PIÙ PRESENTE NELLA PROJECTION
→ cancellata implicitamente

oppure:

ISSUE CHIUSA
→ storico eliminato

Tutti questi comportamenti ridurrebbero la capacità del WCM di ricostruire la propria storia operativa.

49.23 Maturity: che cosa possiamo dire e che cosa no

PROT-020 è **ACTIVE / FIELD VALIDATION**.

Questo significa che esiste una baseline concreta e utilizzabile, ma la maturità della capability non deve essere sovrastimata.

La V1 possiede:

- un modello persistente di issue;
- identità stabile;
- stato **OPEN/CLOSED**;
- criteri di apertura;
- criteri di chiusura;
- projection dedicata;
- confini espliciti rispetto ad authority, Need e workflow.

Non possiede invece, nella baseline descritta dal protocollo:

- un sistema completo di ownership;
- priorità e severity articolate;
- SLA;
- collaborazione tipo ticketing;
- auto-close;
- escalation automatica;
- prova di generalizzazione universale in qualunque contesto operativo.

La maturità corretta è quindi quella di una capability mirata che sta ancora accumulando field evidence.

49.24 In una frase

PROT-020 può essere riassunto così:

Quando una failure tecnica deterministica blocca il WCM, il sistema deve fermarsi in sicurezza e lasciare una traccia durevole, identificabile, verificabile e storicizzata del motivo.

Il fail closed impedisce al sistema di fingere che tutto vada bene.

La technical issue impedisce al sistema di dimenticare perché si è fermato.

Insieme trasformano un errore da evento effimero a parte osservabile della memoria operativa.

Technical Truth Pass — Source Map

Fonti canoniche utilizzate:

- [WCM_AGENT_START.md](#) — source precedence, Persistent Organizational Memory, fail-closed e separazione tra runtime, projection e human view;
- [wcm/documentation/process-memory-book/BOOK_INDEX.md](#) — mapping editoriale CH49 → PROT-020;
- [wcm/process-book/protocols/PROT-020_TECHNICAL_ISSUE_TRACKING_V1.md](#) — fonte tecnica primaria del capitolo.

Maturity qualifier: PROT-020 è **ACTIVE / FIELD VALIDATION**; il capitolo descrive la baseline corrente senza presentarla come sistema di ticketing completo o capability universalmente validata.